

L1 – Introduction & Task Decomposition

Recipe: Task-Decompose an Algorithm

Draw the CFG

nodes = functions/actions; directed edges = data dependency (producer→consumer).

Count distinct actions

= number of functional tasks.

Find independent tasks

no path between them ⇒ may run concurrently.

Try both splittings

functional (by named function, few) vs loop/data (by region, scales – prefer).

State parallelism + joins

how many overlap, where they synchronise.

Recipe: Dependency Analysis

Group

tasks sharing data/constraints.

Order

add constraint where one consumes another's output (+ ghost-exchange before boundary).

Classify shared data

read-only / effectively-local / read-write (accumulate, multi-read-single-write).

Recipe: Map Platform → Terminology

Host (delegates) → Device (receives) → CU → PE → FU. UE = process/thread running a task.

Recipe: Evaluate a Parallel Design

Check: compute > management overhead; #tasks ≥ #PEs; data dependencies minimal; schedule (static if equal, else dynamic / work-stealing).

Sobel Operator

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}; \quad |G| = \sqrt{G_x^2 + G_y^2} \rightarrow |G_x| + |G_y|$$

Ex. S12 – Vocabulary

Explain Host, Device, CU, PE, FU, UE, task.

Host delegates work to a Compute **Device**; device = CU(s) → PE(s) → FU(s). UE = process/thread that runs a task; task = OS process/thread **or** a piece of parallel work.

Ex. S13 – CB1 / Jetson terminology

Programming on the CB1, delegating to the Jetson Nano – what is what?

CB1 = **Host**; Nano = **Device**; Nano SMs = CUs; CUDA cores = PEs; ALU/FPU = FU; spawned threads = UEs; one Sobel slice = task.

Ex. S18 – How does Sobel work?

Determine the edge-detection computation.

Two 3×3 convolutions G_x, G_y give the gradient; magnitude $|G| = \sqrt{G_x^2 + G_y^2}$ **simplified** to $|G_x| + |G_y|$, then threshold.

Ex. S22/23/25 – Pattern language

Name the pattern hierarchy and the Finding-Concurrency patterns.

Hierarchy: Finding Concurrency → Algorithm Structure → Supporting Structure → Implementation Mechanisms. Finding Concurrency = **Decomposition** (task/data) + **Dependency Analysis** (group/order/share).

Ex. S26/27 – Decompose Sobel

Split Sobel into tasks; identify parallelism; functional or loop splitting?

6 functional tasks; sobel_x||sobel_y independent. **Both** apply: functional (limited, 2-way), loop splitting (region-split, scalable – preferred).

Ex. S28/29 – Read the CFG

Distinct actions? Independent tasks? Parallelisation potential? What is a CFG?

6 distinct; only sobel_x,sobel_y independent; functional parallelism limited (2-way), data decomposition scales. CFG = control-flow graph: all execution paths; used by compilers/profilers and to spot concurrency.

Ex. S30–33 – Dependency analysis

Which sub-pattern is most relevant? Group/order/share for Sobel?

Data Sharing is the interesting one. Group: sobel_x/y (same grey input). Order: sobel_all after both finish. Share: grey = read-only, kernel = effectively-local, Gx/Gy = read-write, sobel_all = **accumulate**.

Ex. S34–37 – Evaluate task parallelism

Compute vs management? Tasks vs PEs? Data deps? Schedule/load balancing?

Whole-function task justifies a UE (per-pixel doesn't). Want #tasks ≥ #PEs → region-split. Only dep is the sobel_all join. Equal regions → **static**; else dynamic / work-stealing.

Ex. S38/39 – Fork–Join

What is fork-join? When is it suitable?

Clone code into concurrent tasks, then **join** (wait). **Not** POSIX fork(). Suits recursive/irregular/dynamic task sets; a poor fit for a fixed regular pipeline like Sobel.

Ex. S41–45 – Process vs Thread

What is a process? How do processes/threads differ?

Process: running program; resource + scheduling unit; own main()/address space; MPU-isolated; fork() = CoW clone, parent wait(). **Thread**: inside a process, shares memory, cheap → reuse via **thread pool**.

L2 – Data Decomposition

Recipe: Choose & Size Data Chunks

Feasible? _____

heavy work = data-structure manipulation + similar op applied independently to parts?

Pattern _____

array + local neighbourhood → Geometric Decomposition.

Shape _____

minimise surface/volume → square tiles (strips only for simplicity).

Boundaries _____

add 1-cell ghost halo; exchange a-priori or overlapped.

Map _____

#chunks > #UEs (Distributed Array) for balancing.

Recipe: Surface/Volume Comparison

Volume = cells (computation); Surface = boundary cells (communication). Lower surface/volume wins for equal volume.

Recipe: SPMD Program

Init → get unique ID/rank → common code (branch / index by ID) → distribute data → collect+combine → finalise.

Surface vs Volume

equal volume $N^2/4$: square surface $2N$ vs strip $\frac{5N}{2} \Rightarrow$ **square wins**. 10×10 : square 25/20, strip 20/25.

Ex. S4–5/9 – Patterns recap

What is the data-decomposition path through the patterns?

Finding Concurrency (Decomposition + Dependency) → Algorithm Structure (**Geometric Decomposition**) → Supporting Structure (Distributed Array, SPMD).

Ex. S6–8 – Feasible for edge detection?

Is data decomposition feasible? Tasks? Granularity boundary cases?

Yes: Sobel over every pixel, kernel needs only a 3×3 neighbourhood → regions independent. Tasks = per-region filters. Largest chunk = whole image (no parallelism); smallest = 1 pixel (overhead dominates).

Ex. S10 – Group & order chunks

How to group and order the chunk tasks?

Group tasks on the same chunk. Order: front pipeline first; **ghost-exchange before boundary** updates; `sobel_x/y` → `sobel_all` join per chunk; interior is embarrassingly parallel.

Ex. S12/13 – Square vs rectangle

For a 10×10 matrix in 4 chunks, which shape is better and why?

Equal volume but square surface $2N < \text{strip } \frac{5N}{2} \Rightarrow$ **square** = more compute per unit communication.

Ex. S16/17 – Apply to edge detection

Task parallelism or geometric? What chunks? Data sharing?

Geometric Decomposition; square tiles (or strips); share a 1-pixel **read-only ghost halo**; interior embarrassingly parallel.

Ex. S18–24 – Distributed Array

How are chunks mapped to UEs for load balance?

Make #chunks > #UEs; assign block-cyclic / round-robin so each UE owns several scattered chunks → smoother load; localise + halo per UE.

Ex. S27/28 – SPMD evaluated

Strengths and limits of SPMD?

One maintainable, portable code base; overhead only at start-up/shutdown; aligns with message passing. **Limit:** branching into genuinely different per-UE tasks stretches the pattern; needs good data management.

L3 – OpenMP

Recipe: Predict OpenMP Behaviour

Read directives

`parallel` alone = whole block per thread (replicate); `parallel for` = iterations split.

Scope each var

`shared` (race risk) / `private` (uninit, discarded) / `firstprivate`/lastprivate / `reduction` (race-free).

Find races

`shared` var written by > 1 iteration → indeterminate.

Count

multiply by thread count only in replicated regions.

Recipe: Pick a Sync Primitive

one block at a time → `critical`; single memory op → `atomic`; accumulate → `reduction`; phase wall → `barrier`; cross-process → `semaphore`.

Recipe: Choose a Schedule

even iterations → `static`; uneven → `dynamic`; tapering → `guided`.

Ex. S7 – parallel vs parallel for

With T threads, $n = 5$: how many prints for `parallel` alone vs `parallel for`?

$T \times 5$ (loop replicated) vs 5 (iterations shared). `parallel` makes threads; `for` splits work.

Ex. S8 – shared vs private

`shared(b)` loop then `private(b)` loop, both $b=a+i$. Final b ?

S1 `shared`: **race** → any of 7..16. S2 `private`: copies discarded → global b unchanged. Final = whatever S1 left.

Ex. S18 – Barrier internals

Spin or block on arrival? Who runs first on release?

Depends: short + spare cores → spin (active); long/oversubscribed → block (passive). Release order = **OS, nondeterministic**; use `ordered` if needed.

Ex. S21 – Reduction sum

How to sum an array race-free; what does `reduction` do?

`#pragma omp parallel for reduction(+:sum)`: each thread keeps a **private partial**, combined after the join.

Ex. S22 – nowait

Effect of `nowait` on `single`?

`single` = one thread runs the block (implicit barrier). `nowait` **removes the barrier** → others proceed (safe only if no dependency).

Ex. S28 – Thread count

What determines the number of threads?

Precedence: `if` → `num_threads` → `omp_set_num_threads` → `OMP_NUM_THREADS` → default ($\approx$ #CPUs).

L4 – SIMD (Flynn, NEON, MISD)

Recipe: Decode a NEON Intrinsic

Pattern `v <op> q? _ <type>`: **op** = operation; **q** present $\Rightarrow$ 128-bit Q else 64-bit D; **type** `<base><bits>x<count>_t` $\rightarrow$ **count** = lanes. Lanes = $\frac{\text{reg width}}{\text{lane bits}}$.

Recipe: Classify with Flynn

instruction streams $\times$ data streams: 1/1 SISD, 1/many SIMD, many/1 MISD, many/many MIMD.

Ex. S6/9 – SPMD pattern

What is SPMD and why is it here?

One program per UE, unique ID, branch/index by ID. It is the software generalisation of **SIMD** (same instruction, different data).

Ex. S10/17 – SIMD motivation & NEON

Why vectorise? What is NEON?

One instruction processes many lanes $\rightarrow$ throughput. NEON = ARM SIMD; **Q**=128-bit ($2 \times 64 / 4 \times 32 / 8 \times 16 / 16 \times 8$), **D**=64-bit; built on **MAC**, no divide.

Ex. S18/19 – Decode intrinsics

`vmulq_f64?` lanes in `uint16x4_t?` `vmul_lane_u16(a,b,L)?`

`vmulq_f64`: 128-bit lane-wise multiply of **2 doubles** $\rightarrow$ `float64x2_t`. `uint16x4_t`: **4 lanes**. `vmul_lane`: broadcast `b[L]` to all lanes, multiply by `a`.

Ex. S20 – Lab scenario (calculate)

Filtering with NEON: how many lanes/iterations to process *K* elements?

Lanes per op = vector width / element size; iterations = $\lceil K / \text{lanes} \rceil$. E.g. `uint16x8` on *K* pixels $\rightarrow \lceil K / 8 \rceil$ vector ops (+ MAC for the filter).

Ex. S21–24 – MISD / lockstep

What is MISD? Real-world example?

Many instructions on one data stream, for **dependability** not speed. **U96 / Zynq ZU3EG** $2 \times$ Cortex-R5F lockstep: compare bus $\rightarrow$ RESET on mismatch (e.g. ABS). Loose = dissimilar impls + acceptance voting; **2oo3** tolerates one fault.

L5 – Operating Systems

Recipe: Apply Amdahl's Law

Find p (parallel fraction) $\rightarrow S(N) = \frac{1}{(1-p)+p/N} \rightarrow \text{ceiling } S_{\max} = \frac{1}{1-p} \ (N \rightarrow \infty)$.

Recipe: Reason About MP Scheduling

Need balance $\rightarrow$ SQMS; need cache affinity $\rightarrow$ MQMS; imbalance $\rightarrow$ migration / work-stealing.

Recipe: Pick a Linux Scheduler

hard deadline $\rightarrow$ DEADLINE; strict priority $\rightarrow$ FIFO/RR; fairness $\rightarrow$ OTHER (EEVDF); background $\rightarrow$ BATCH/IDLE.

Recipe: Build a Lock

atomic op (TAS) $\rightarrow$ spin (cheap read first = TTAS) $\rightarrow$ or block via OS mutex/semaphore (no spin).

Amdahl's Law

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\max} = \frac{1}{1-p}$$

Ex. S6/7/9 – Scheduling & categories

Explain SQMS/MQMS and the three scheduler categories.

SQMS: 1 queue (simple, contention, cache bounce). MQMS: per-CPU (**cache affinity**, imbalance). Categories: **RT** (FIFO/RR/DEADLINE), **Fair** (OTHER=EEVDF, RB-tree on `vruntime`, `nice`), **Low** (BATCH/IDLE).

Ex. S8 – Amdahl worked

$p = 0.75$: speed-up on 4 cores? maximum?

$$S(4) = \frac{1}{0.25+0.1875} \approx 2.29\times; S_{\max} = \frac{1}{0.25} = 4\times.$$

Ex. S11 – Critical section

What is a critical section and how is entry made atomic?

RW-shared data; one task inside at a time. Atomicity: disable IRQ · ISA atomic · Peterson/spinlock · language · OS mutex/semaphore (**blocks, no spin**).

Ex. – TAS vs TTAS

Why is TTAS faster under contention?

TAS spins with atomic **writes** $\rightarrow$ bus lock + invalidation. TTAS spins on a cached **read**, writes only when the lock looks free $\rightarrow$ far less coherence traffic, near-ideal.

L6 – Memory

Recipe: Trace a Memory Access

Pipeline
stage requests an address (IF / MEM).
Cache
hit → serve fast (done). Miss → continue.
MPU
check access rights (r/w/x allowed?).
MMU / TLB
TLB hit → frame #. Miss → walk page directory/table.
Page fault
empty entry → MM software loads page from disk into a free frame, updates page directory.
Fill & cache
TLB stores logical→frame; cache reads a whole **line**; a write sets the **dirty bit**. Next nearby access = hit.

Recipe: Will a Cache Writeback Happen?

A dirty line is written back when (1) the controller decides to, or (2) it needs the line (eviction). After writeback dirty→0; on eviction valid→0, refill, valid→1.

Recipe: Diagnose Coherence / False Sharing

Same line in ≥2 caches + a write? → invalidation traffic. **True** sharing = same var; **false** sharing = different vars, same line → pad/align to separate lines.

Recipe: Classify a Memory Architecture

equal access → UMA (~4-CPU limit); fast local + slow remote via interconnect → NUMA (ccNUMA); interleaved to fake contiguous → SUMA.

Ex. S4 – Explain cache

What is a cache and what happens on hit/miss?

Fast memory near the CPU, **transparent** to the programmer. Hit → served fast; miss → fetch a **line** from memory (also stored in cache for locality).

Ex. S5 – Explain paging

What is paging? Resident vs working set?

Address space → pages (4K/2M); physical → frames; load only used pages (relative addressing → any frame). **Resident set** = pages in memory; **working set** = pages in use now.

Ex. S6–12 – Trace the code (worked)

Walk mov ea,var1; mov eb,var2; add ea,eb; mov var3,ea through the system. Why is the first fetch slow but the next fast?

First fetch (0x100): cache miss → MPU → MMU → TLB miss → page-walk empty → **page fault** (MM loads frame #2). Fill brings a whole line (l1–l4) + caches TLB mapping ⇒ 0x104,0x108 are **cache hits** inside the CPU. Data var1 (0x400): second page fault (frame #8); var2 = hit. mov var3 = write → sets **dirty bit**.

Ex. S13 – Two types of thrashing

Explain cache vs disk thrashing.

Cache: many memory lines map to the same cache line → load A, load B (same slot, evicts A), need A again → repeated misses. **Disk**: too few frames → each page request evicts another (dirty → write back first) → stall on slow disk.

Ex. S14/15 – Context-switch cost

Cost of NOT programming cache-aware? Per-component switch cost?

Cache **high** (flush + write-back dirty), TLB **high** (flush); MPU/page-tables medium; pipeline low. CPI spikes after a switch, falls as cache/TLB re-warm.

Ex. S16 – Cache writeback

When and how does a dirty line get written back?

(1) controller decides, or (2) it needs the line and it's dirty → write back first. Then dirty→0; eviction may set valid→0, refill, valid→1.

Ex. S17/18 – Multicore L1/L2

Explain L1 read and write-back on a multicore with shared L2.

Read: line pulled L2→L1 of the requesting core. Write-back: **flushing L1 requires flushing L2**; L1→L2 write-back may or may not trigger an L2→memory write-back; coherence keeps other L1s consistent.

Ex. S19/20 – Coherence vs consistency

Define both; what does the 0-1-1-2 / 0-2-1-2 example show?

Coherence: a write **will** become visible (order to one location preserved); not in ISA; necessary not sufficient. The two read sequences show two cores observing writes in **different** order → a protocol must make P2 see P1's order. **Consistency**: ordering across **different** locations – sequential / processor / release (barriers).

Ex. S21/22 – Snooping vs directory

Outline both coherence protocols.

Snooping: bus broadcast; controllers listen, invalidate/update; non-scalable (e.g. P1 writes var1 → P2 copy invalidated). **Directory**: point-to-point; owner invalidates sharers, waits for ack, then writes and becomes owner; scalable.

Ex. S23 – False sharing

Two cores write var3[0] and var3[1] (same line). Problem and fix?

False sharing: each write invalidates the other's copy → line ping-pongs (useless coherence traffic). **Fix**: pad/align so the vars sit on separate cache lines.

Ex. S24 – ASID / CPU pinning

How is expensive cache/TLB state preserved across switches?

Physical-address tagging or an **ASID** lets one process's lines survive while another runs ⇒ cheap to re-schedule on the same core (**CPU pinning** / **processor affinity**).

Ex. S25–28 – UMA / NUMA / SUMA

Limits of the traditional bus? Explain UMA, NUMA, SUMA.

Bus latency grows with length → doesn't scale (~4 CPUs). **UMA**: equal access, L2/L3 buffer, linkable blocks. **NUMA**: fast local memory, all reachable via interconnect (slow remote), ccNUMA protocol. **SUMA**: interleaving → best-case UMA.

Ex. S29 – OS support for NUMA
How does Linux handle NUMA?

Finds HW **cells** → maps to **nodes**; keeps work + memory local (cache affinity, less interconnect traffic); per-node page tables + **zonelists** by NUMA distance; tools `taskset/numactl/sched_setaffinity`.

L7 – CPU Architectures

Recipe: Classify a CPU Memory Architecture

shared I+D bus → Von Neumann; fully separate → Harvard; split L1 I/D but unified L2/RAM → **Modified Harvard** (e.g. A53); DSP separate program/data bus → Harvard.

Recipe: Identify the Parallelism Mechanism

multiple FUs (VLIW) = data; multiple pipelines (superscalar) = instruction; spare slots filled by 2nd thread = SMT; separate cores = multicore/AMP.

Recipe: Set Up AMP

same ISA → SMP/big.LITTLE one OS; different ISA → separate OS/bare-metal per core + openAMP (remoteproc boots ELF, rpmsg/virtio comms).

Ex. S4–7 – GP-CPU architectures

Von Neumann vs Harvard vs Modified Harvard? Ways to parallelise a CPU?

Shared / separate / split-L1+unified-L2. Parallelise via: multicore, multiple FUs (data), multiple pipelines (instruction), heterogeneous PEs.

Ex. S5 – A53 architecture

Separate L1-I/L1-D, unified L2 – which architecture?

Modified Harvard: Harvard inside (split L1), Von Neumann outside (unified L2/memory).

Ex. S8–10 – DSP / VLIW

What memory architecture? How does execution work?

Harvard (separate program/data bus keeps MAC fed). **VLIW**: compiler packs one op per FU (.M/.D/.L/.S) into a wide word, run in parallel (+ software pipelining). Big code, binary-incompatible.

Ex. S11–14 – Pipelines

Hazards? Super-pipeline vs superscalar? Issue/execute order?

Hazards: structural/data/control. **Super-pipeline** = finer stages (issue faster); **superscalar** = more pipelines (several/cycle). Order: in-order → OoO exec → **OoO issue+exec**.

Ex. S15–18 – Co-processors

How does a co-processor work and what are its drawbacks?

CPU passes unrecognised instructions to it; CPU **stalls** (8087 + 8086 on FMUL); own ISA/clock/mem (NEON). Drawbacks: expensive decoders, integration cost, compiler burden.

Ex. S19 – Why SMT needs superscalar

Why does SMT require a superscalar core?

SMT fills **idle issue slots** with a 2nd thread; only superscalar cores **have** multiple slots/pipelines per cycle. ~30% gain (cache-stall dependent); security issues.

Ex. S20–25 – AMP / openAMP

AMP scenarios? Different ISA under one OS? How do cores communicate?

Scenarios: diff/same ISA, diff clocks, big.LITTLE, diff/no OS. Different ISA under one OS = **No** → separate OSes + **openAMP**: remoteproc boots the ELF, rpmsg/virtio ring buffers carry messages. Zynq: 4×A53 SMP + 2×R5 lockstep + FPGA.

L8 / 8a – GPU, CUDA & OpenCL

Recipe: Size a CUDA / OpenCL Launch

one thread per element $\rightarrow$ `#threads = #elements`; pick `threadsPerBlock` (multiple of 32 = warp); grid = `[elements/block]`; 2-D `[R][C]` $\rightarrow$ `threadIdx.x` $\in$ `0..R-1`, `.y` $\in$ `0..C-1`.

Recipe: Convert a Loop to a Kernel

drop the loop $\rightarrow$ `i=get_global_id(0)` (CUDA `threadIdx/blockIdx`) $\rightarrow$ body on element `i`; work-items must be independent.

Recipe: Map CUDA $\leftrightarrow$ OpenCL

`thread/block/grid/SM` $\equiv$ `work-item/work-group/NDRange/CU`; `host` $\rightarrow$ `device` $\rightarrow$ `CU` $\rightarrow$ `PE`.

Ex. S4/5 – GPU vs CPU

How do GPU and CPU differ; thread/loop relationship?

CPU = latency-optimised (few fast threads); GPU = **throughput** (many slow threads). Each loop iteration $\rightarrow$ one GPU thread.

Ex. S6/7 – Thread & block organisation

Into how many threads is a `[10][5]` op divided? `threadID` range?

One per element $\rightarrow$ **`10  $\times$  5 = 50 threads`**; `threadIdx.x` $\in$ `0..9`, `threadIdx.y` $\in$ `0..4` (IDs (0,0)..(9,4)). Threads grouped into blocks, blocks into a grid.

Ex. S8/9 – Execution model / SM

What is an SM? How are blocks executed?

SM = Streaming Multiprocessor; a block maps to an SM, which runs threads in **warps of 32**. The grid of blocks is distributed across SMs (queue the rest). Size blocks $\times$ 32.

Ex. S10/11 – Limitations

Why not one thread per pixel for everything?

Divergent branches serialise a warp; memory coalescing matters; too-fine work wastes scheduling. Best with regular, independent, arithmetic-heavy data.

Ex. S14/15 – Rendering pipeline

Name the stages; why is it so parallel?

world coords $\rightarrow$ vertex $\rightarrow$ assembly $\rightarrow$ (geometry) $\rightarrow$ rasterise $\rightarrow$ fragment (per pixel) $\rightarrow$ blend. Data **expands** few vertices $\rightarrow$ millions of pixels = massively parallel per-pixel work.

Ex. S21–26 – OpenCL models

Name the four models and the memory regions.

Platform / Execution / Memory / Programming. `work-item/work-group/NDRange`; **weak consistency**; regions Global / Constant / Local / Private.

Ex. S27/28 – Loop $\rightarrow$ OpenCL kernel

Rewrite `for(i<4096) c[i]=a[i]+b[i]`; and give the `NDRange`.

```
1 __kernel void vadd(__global float*a,*b,*c){
2   int i=get_global_id(0); c[i]=a[i]+b[i]; }
NDRange = 4096 work-items (e.g. work-group 512); loop replaced by the index space.
```

Ex. S29 – Coarse vs fine scheduling

Two scheduling levels; why interleave warps?

Coarse: blocks $\rightarrow$ SMs. **Fine**: warp scheduler issues 32-thread warps, dual-issue, **interleaves** – when one stalls on memory, another runs $\rightarrow$ **hides latency**.

Ex. S31–33 – OpenCL queue & sync

Command-queue ordering? Where can you synchronise?

Host enqueues in order; completion in/out-of-order. Sync **only within a work-group** (barrier, fence, atomics) – none between work-groups.

Ex. S40/41 – AMP recap

Different ISA under one OS?

No – separate OSes coupled by openAMP (`remoteproc + rpmsg/virtio`). Same as L7.

L9 – MPI (Message Passing Interface)

Recipe: Write an MPI Program

Init → Comm_size/Comm_rank → branch by rank (SPMD) → Send/Recv or collectives → Finalize. Build mpicc, run mpirun -n X.

Recipe: Choose a Send Mode

guaranteed rendezvous → Ssend; decouple sender → Bsend; recv already posted → Rsend; else Send; overlap → Isend/Irecv+Wait/Test.

Recipe: Pick a Collective

same data to all → Bcast; split out → Scatter; bring back → Gather; combine → Reduce/Allreduce; phase wall → Barrier.

Ex. S2/6 – Distributed architectures
Tightly vs loosely bound? Why MPI?

Tight = shared memory over interconnect; loose = distributed memory (MPI’s home: SPMD→MIMD). Protocols rise: variable → scatter-gather → RPC → distributed objects.

Ex. S7–10 – MPI basics
Six fundamental operations? What is a communicator/rank?

Init,Comm_size,Comm_rank,Send,Recv,Finalize. Communicator (MPI_COMM_WORLD) groups processes; each has a unique rank used to branch/index.

Ex. S11–13 – Build & run
How to compile/launch SPMD vs MIMD?

mpicc (gcc wrapper). SPMD: mpirun -n X ./p (X>cores = oversubscribe). MIMD: mpirun -n 2 p1 : -n 2 p2. Cluster: ssh + hostfile; bind -cpu-set.

Ex. S14 – Blocking vs non-blocking
Difference, and why it matters?

Send/Recv block until buffer empty / data in. Isend/Irecv return immediately → Wait/Test (overlap compute + comms). Communication is the synchronisation.

Ex. S15/16 – Four send modes
Name the four blocking send modes.

Standard (Send) · Synchronous (Ssend, safest) · Buffered (Bsend, app buffer) · Ready (Rsend, lowest overhead, recv pre-posted).

Ex. S17–19 – Message anatomy
What’s in a message? Derived datatypes?

Envelope (source, dest, communicator, tag) + Body (count, datatype, buffer). Standard types MPI_INT/FLOAT/...; build structs via MPI_Type_create_struct.

Ex. S20 – Collectives
Explain broadcast / scatter / gather.

Bcast: root→all (tree). Scatter: chunks out to each. Gather: chunks back to root. Map to Distributed Array / Geometric Decomposition.

Ex. S21 – Reduction
Where have we seen reduction? What does MPI_Reduce do?

OpenMP reduction (L3) and accumulate sharing (L1–2). MPI_Reduce(send,recv,count,type,op,root,comm) combines element-wise across procs (SUM/MAX/MIN/PROD/...); Allreduce → all.

Ex. – Pick a send mode
Must be sure the receiver has started before continuing – which send?

MPI_Ssend (synchronous): completes only once the matching receive has begun.

Cross-lecture threads: Reduction OpenMP(L3)↔accumulate(L1-2)↔MPI_Reduce(L9) · SPMD pattern(L2)←SIMD/SIMT(L4)→MPI(L9)→CUDA/OpenCL(L8) · Scheduling L1→L3→L5→L8 · Affinity MQMS(L5)↔ASID(L6)↔-cpu-set(L9) · Amdahl(L5) caps all.